

Practical-08

Name: Simran Jangid

Rollno: 2520030166

Section: 8

EXPERIMENT – Dynamic Memory Allocation and Copy-on-Write

Part A: Dynamic Memory Allocation Using malloc(), calloc(), realloc(), and free()

Aim: To develop a C program using malloc(), calloc(), realloc(), and free(), and identify memory leaks using **Valgrind**.

Objective:

- To understand dynamic memory allocation.
- To use malloc(), calloc(), realloc(), and free().
- To monitor allocated memory.
- To detect memory leaks using Valgrind.

Theory: Dynamic memory is allocated from the **heap** during program execution.

Function	Purpose
malloc()	Allocates uninitialized memory
calloc()	Allocates and initializes memory to zero
realloc()	Changes the size of allocated memory
free()	Releases allocated memory

Algorithm:

1. Allocate memory using malloc().
2. Allocate memory using calloc().

3. Increase memory using `realloc()`.
 4. Display the allocated values.
 5. Release memory using `free()`.
 6. Run the program using Valgrind.
 7. Check the memory-leak report.
-

C Program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    // malloc()
```

```
    int *a = (int *)malloc(5 * sizeof(int));
```

```
    if (a == NULL)
```

```
    {
```

```
        printf("malloc failed\n");
```

```
        return 1;
```

```
    }
```

```
    for (i = 0; i < 5; i++)
```

```
        a[i] = i + 1;
```

```
printf("malloc values: ");  
for (i = 0; i < 5; i++)  
    printf("%d ", a[i]);  
  
printf("\n");  
  
// calloc()  
int *b = (int *)calloc(5, sizeof(int));  
  
if (b == NULL)  
{  
    printf("calloc failed\n");  
    free(a);  
    return 1;  
}  
  
printf("calloc values: ");  
for (i = 0; i < 5; i++)  
    printf("%d ", b[i]);  
  
printf("\n");  
  
// realloc()  
a = (int *)realloc(a, 10 * sizeof(int));
```

```
if (a == NULL)
{
    printf("realloc failed\n");
    free(b);
    return 1;
}

for (i = 5; i < 10; i++)
    a[i] = i + 1;

printf("After realloc: ");
for (i = 0; i < 10; i++)
    printf("%d ", a[i]);

printf("\n");

// free()
free(a);
free(b);

printf("Memory released successfully.\n");

return 0;
}
```

OUTPUT:

```
simra@simran315:~$ nano memory_allocation.c
simra@simran315:~$ gcc memory_allocation.c -o memory_allocation
simra@simran315:~$ ./memory_allocation
malloc values: 1 2 3 4 5
calloc values: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
Memory released successfully.
```

Observation:

Valgrind showed that all dynamically allocated memory was released using `free()`. Therefore, there were **no memory leaks**.

Result: The program successfully demonstrated `malloc()`, `calloc()`, `realloc()`, and `free()`. Valgrind was used to verify that the allocated memory was properly released.

Part B: Copy-on-Write Using `fork()`

Aim: To demonstrate **Copy-on-Write (COW)** behaviour after `fork()` and analyse memory usage before and after modifying data in the child process.:

Objective:

- To understand Copy-on-Write.
- To observe memory sharing after `fork()`.
- To analyse memory before and after modification by the child.

Theory:

When `fork()` creates a child process, Linux initially allows the parent and child to **share the same physical memory pages**.

If the child modifies a shared page, Linux creates a **separate copy** of that page for the child.

This is called **Copy-on-Write**.

Before `fork`:

Parent
|
Memory
|
Page A

After fork:

Parent	Child
+-----+-----+	
Shared Page A	

After Child modifies:

Parent	Child
Page A	Page B
(Copy)	

Algorithm:

1. Allocate a large memory area.
 2. Fill it with data.
 3. Display the process PID.
 4. Create a child using fork().
 5. Both processes initially share the memory pages.
 6. Display memory information.
 7. Modify the memory in the child.
 8. Display memory information again.
 9. Observe the increase in the child's private memory.
-

C Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
#define SIZE (50 * 1024 * 1024)
```

```
int main()
```

```
{
```

```
    char *memory;
```

```
    memory = malloc(SIZE);
```

```
    if (memory == NULL)
```

```
{  
    printf("Memory allocation failed\n");  
    return 1;  
}  
  
// Touch the memory so physical pages are allocated  
for (int i = 0; i < SIZE; i += 4096)  
    memory[i] = 'A';  
  
printf("Parent PID: %d\n", getpid());  
printf("Memory allocated: 50 MB\n");  
  
pid_t pid = fork();  
  
if (pid < 0)  
{  
    perror("fork failed");  
    free(memory);  
    return 1;  
}  
  
if (pid == 0)  
{  
    printf("\nChild PID: %d\n", getpid());  
    printf("Child created. Memory is initially shared.\n");
```

```
sleep(10);
```

```
printf("Child modifying memory...\n");
```

```
for (int i = 0; i < SIZE; i += 4096)
```

```
    memory[i] = 'B';
```

```
printf("Child modified the memory.\n");
```

```
sleep(20);
```

```
free(memory);
```

```
exit(0);
```

```
}
```

```
else
```

```
{
```

```
    printf("Parent created child PID: %d\n", pid);
```

```
    printf("Parent waiting...\n");
```

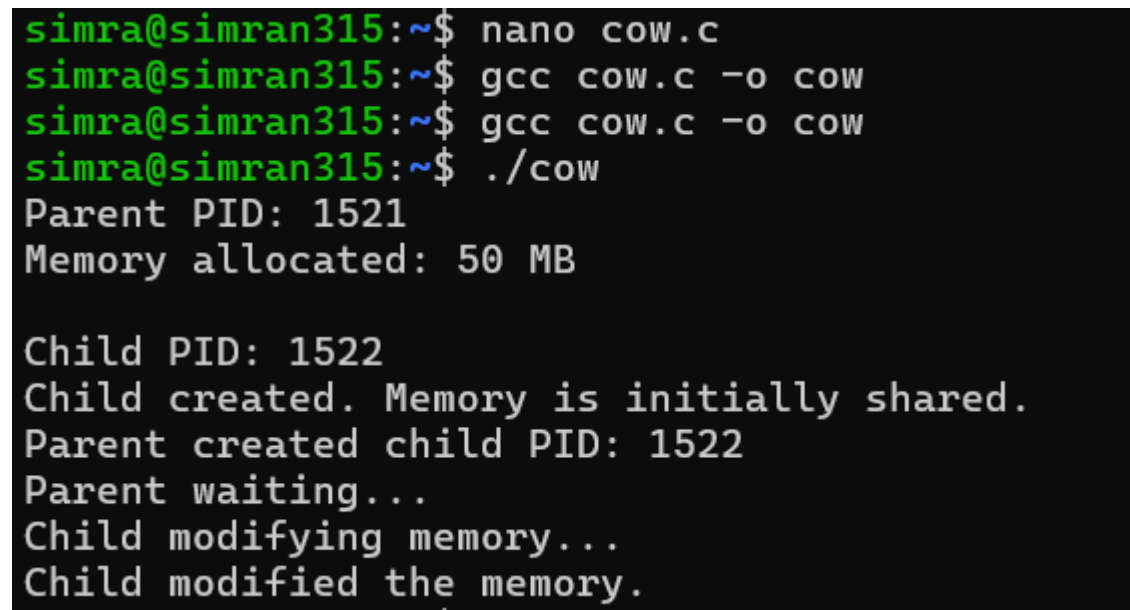
```
sleep(30);
```

```
wait(NULL);
```

```
free(memory);
```

```
}  
  
return 0;  
  
}
```

OUTPUT:



```
simra@simran315:~$ nano cow.c  
simra@simran315:~$ gcc cow.c -o cow  
simra@simran315:~$ gcc cow.c -o cow  
simra@simran315:~$ ./cow  
Parent PID: 1521  
Memory allocated: 50 MB  
  
Child PID: 1522  
Child created. Memory is initially shared.  
Parent created child PID: 1522  
Parent waiting...  
Child modifying memory...  
Child modified the memory.
```

Observation:

Before child modification

The parent and child initially share the physical memory pages created before fork().

After child modification

When the child writes to the memory, Linux creates private copies of the affected pages for the child.

Therefore:

Before modification:

Parent → Shared Pages ← Child

After modification:

Parent → Original Pages

Child → Copied Pages

The child's private memory usage increases as the shared pages are modified.

Result: The Copy-on-Write behavior of `fork()` was successfully demonstrated. Initially, the parent and child shared physical memory pages. After the child modified the memory, Linux created private copies of the modified pages.

Conclusion: The experiment demonstrated two important Linux memory-management concepts:

1. **Dynamic memory allocation** using `malloc()`, `calloc()`, `realloc()`, and `free()`, with Valgrind used to detect memory leaks.
2. **Copy-on-Write** after `fork()`, where memory pages are initially shared and copied only when one process modifies them.